

Agile Software Development

1. Agile Development Model

1.1 What is Agile?

Agile is a **lightweight, adaptive development approach** where software is delivered in **small usable increments** and requirements evolve through continuous collaboration between teams and stakeholders.

1.2 Agile vs Traditional (Waterfall)

Aspect	Traditional	Agile
Delivery	One final release	Frequent small releases
Requirements	Fixed early	Evolving
Planning	Detailed upfront	Adaptive
Feedback	Late	Continuous
Change handling	Difficult	Welcomed
Testing	After coding	Continuous

2. Agile Manifesto — Principles

Agile values:

- Individuals & interactions over processes & tools
- Working software over documentation focus
- Customer collaboration over contract negotiation
- Responding to change over following a rigid plan

Supporting principles include:

- Deliver working software frequently
- Welcome changing requirements
- Close daily collaboration
- Self-organizing teams
- Continuous improvement

3. Agile Components & Benefits

3.1 Key Components

- Iterations / Sprints
- User Stories
- Backlogs (Product & Sprint)
- Daily Stand-ups
- Reviews & Retrospectives

- Continuous Integration & Testing
-

3.2 Benefits of Agile

Agile provides:

- Faster delivery of value
 - Early validation
 - Improved quality
 - Transparency
 - Risk reduction
 - Better stakeholder satisfaction
 - High adaptability
 - Motivated teams
-

4. Agile Tools

Common Agile tools include:

- Jira
- Azure DevOps
- Trello
- Rally
- ClickUp
- Asana

These tools help manage backlogs, sprints, tasks, burndown charts, and collaboration.



Agile Frameworks

Agile is an **umbrella term** that includes multiple lightweight development methodologies that follow Agile values and principles.

The most commonly used Agile frameworks are **Scrum, Extreme Programming (XP), and Kanban/Lean.**

5.1 Scrum

Scrum is the **most widely used Agile framework.** Work is delivered in short, fixed-length cycles called **Sprints.**

Scrum Roles

Role	Responsibility
Product Owner	Owns the product backlog, defines priorities, and ensures the team delivers the highest-value features first.
Scrum Master	Facilitates Scrum practices, removes impediments, coaches the team, and ensures the Scrum process is followed.
Development Team	Cross-functional team responsible for developing and delivering usable product increments every sprint.

Scrum Artifacts

Artifact	Description
Product Backlog	Ordered list of all desired features, enhancements, and fixes for the product. It evolves continuously.
Sprint Backlog	Subset of Product Backlog items selected for the current Sprint, along with a plan to deliver them.
Increment	The working, tested, usable product output delivered at the end of a Sprint.

Scrum Events (Ceremonies)

Event	Purpose
Sprint Planning	Define Sprint goals and select backlog items to work on.
Daily Scrum (Stand-up)	A short 15-minute meeting to review progress, discuss blockers, and plan the next 24 hours.
Sprint Execution	Period where development and testing are carried out to complete Sprint work.
Sprint Review	Demonstration of the completed Increment to stakeholders for feedback.
Sprint Retrospective	Internal team meeting to discuss what went well, what needs improvement, and how to improve in the next Sprint.

Sprint Duration

- Each Sprint normally lasts **1 to 4 weeks**.
 - The duration remains **fixed and consistent across Sprints** for a given project.
-

5.2 Extreme Programming (XP)

Extreme Programming focuses strongly on **technical excellence and high code quality**.

Key XP Practices

- **Test-Driven Development (TDD)** — write tests before code
- **Pair Programming** — two developers work together at one workstation
- **Continuous Integration** — frequent code integration and testing
- **Simple Design** — keep design as simple as possible
- **Refactoring** — continuously improve code structure
- **Collective Code Ownership** — any developer can improve any part of the code
- **Coding Standards** — consistent style and readability

Goal of XP

XP ensures that software is:

- ✓ Clean
 - ✓ Well-tested
 - ✓ Maintainable
 - ✓ Adaptable to change
- 
-

5.3 Kanban / Lean

Kanban → Kanban manages work using a **visual workflow board** showing task status.

Example workflow:

To Do → In Progress → Testing → Done

Key focus:

- Limit Work-In-Progress (WIP)
 - Prevent overload
 - Improve flow efficiency
-

Lean Software Development

Lean emphasizes:

- **Waste reduction** — remove activities that do not add value
- **Flow optimization** — smooth and continuous delivery
- **Value focus** — deliver only what benefits the customer

Lean thinking improves **speed, efficiency, and quality**.

6. Jira Usage in Agile — Clarified Notes

Jira is a **widely used Agile project management tool** that helps teams plan, track, and manage software development work in an organized manner.

It supports Agile practices such as **backlogs, sprints, task tracking, dashboards, and reports**.

6.1 Projects in Jira

In Jira, a **Project** represents:

A software product

or

A workspace for a specific team

Each project contains:

- **Backlogs** — list of all work items
- **Issues** — individual tasks, bugs, or stories
- **Sprints** — short development cycles
- **Reports** — progress and performance analytics

Projects act as the **central container** for all Agile activities.

6.2 Tasks & Subtasks in Jira

Task

A **Task** is a unit of work assigned to a team member — for example coding, testing, or documentation work.

Subtask

A **Subtask** is a **smaller piece of work derived from a Task**.

It helps break down large work items into manageable actions.

Other Common Jira Issue Types

Issue Type	Meaning
Story	Represents user-facing functionality or a business requirement.
Bug	Represents a defect that needs fixing.
Epic	A large feature or objective that is broken down into multiple Stories.

These issue types help in **classifying and grouping work logically**.

6.3 Sprints in Jira

A **Sprint** is a fixed-length iteration (usually 1–4 weeks) during which selected work items are completed.

Sprint Workflow in Jira

1. **Product Owner prioritizes the backlog**
Highest-value work is placed at the top.
2. **Sprint Planning — team selects work**
Tasks / Stories are moved into the Sprint backlog.
3. **Tasks are distributed to team members**
Responsibility is assigned.
4. **Daily Scrum updates take place**
Short daily sync meeting.
5. **Development & testing are executed**
Work is completed collaboratively.
6. **Sprint Review is conducted**
The increment is demonstrated to stakeholders.
7. **Sprint Retrospective is held**
Team reflects and identifies improvements.
8. **Sprint is closed & Increment is released**
The working output becomes part of the product.

Jira Features Supporting Sprints

Jira provides:

- **Sprint Boards** — visualize progress (To-Do → In-Progress → Done)
- **Burndown Charts** — show remaining work vs time
- **Velocity Tracking** — measures work completed per sprint

These help teams:

- ✓ Monitor progress
 - ✓ Forecast capacity
 - ✓ Improve planning accuracy
-

7. Agile Web Application — Case Study Flow

Example System: **Online Shopping Web Application**

This case study explains **how Agile is applied in real projects** using sprints, backlogs, and continuous feedback.

7.1 Product Backlog — Example Items

The **Product Backlog** contains all required features, prioritized by the Product Owner.

Example backlog items:

- User login & authentication
- Browse products
- Shopping cart
- Online payment
- Order history

Each backlog item will later be broken into **user stories and tasks**.

7.2 Sprint Execution — Example Breakdown

The project is divided into multiple **Sprints**, each delivering a usable Increment.

Sprint 1 — Deliverables

Deliver:

- ✓ User Login
- ✓ Product Listing

Activities performed:

- Create user stories
- Estimate effort
- Assign tasks to team members
- Design UI and backend logic
- Perform unit and integration testing
- Demonstrate completed functionality to stakeholders

At the end of Sprint 1:

A working product with **login + product viewing** is delivered.

Sprint 2 — Deliverables

Deliver:

- ✓ Shopping Cart
- ✓ Checkout Process

This sprint enhances the application to allow users to:

- Select items
 - Review cart
 - Proceed to checkout
-

Sprint 3 — Deliverables

Deliver:

- ✓ Online Payment
- ✓ Order History

System now supports:

- Secure payments
- Viewing past orders

By the end of Sprint 3, the main shopping workflow is complete.

7.3 Continuous Feedback Loop

After every Sprint:

- ✓ Stakeholders review delivered features
- ✓ Feedback is collected
- ✓ Product Backlog is refined or reprioritized
- ✓ Team reviews & improves its working process
- ✓ An updated version of the application is released

This ensures **continuous improvement and high business alignment**.

8. Agile Testing & Quality Alignment

In Agile, **testing is not a separate phase — it happens inside every Sprint.**

This includes:

- **Unit Testing** — testing individual functions/modules
- **Integration Testing** — checking interactions between modules
- **System Testing** — validating complete system behavior
- **User Acceptance Testing (UAT)** — validating business needs
- **Automation Testing (where suitable)** — repeated tests scripted

Key Principle

Testing happens **early and continuously**, ensuring:

- ✓ Early defect detection
 - ✓ Stable working increments
 - ✓ Reduced risk at release time
-